

MINIMUM MALİYETLİ YAYILAN AGAC

Spanning Tree · Kruskal Algoritması · Union-Find · Karmasiklik Analizi

Spanning Tree nedir? Bir grafın tüm düğümlerini içeren ve $|V| - 1$ kenar içeren, dongsuz alt grafıdır. **MST (Minimum Spanning Tree):** Kenar ağırlıklarının toplamı en küçük olan spanning tree.

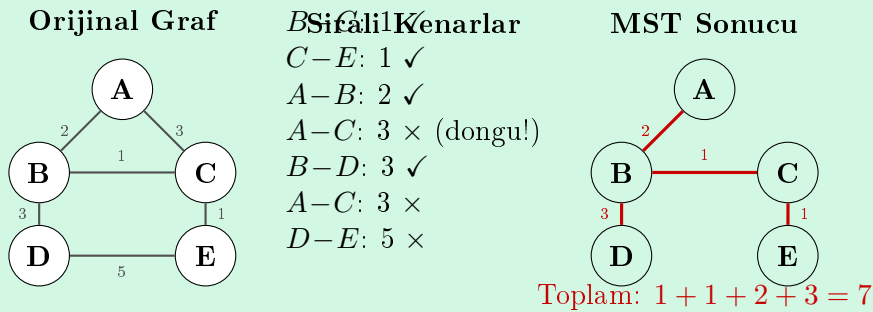
1. Minimum Maliyetli Yayılan Agac – Temel Özellikler

Minimum Maliyetli Yayılan Agac (Spanning Tree)

Spanning Tree özellikleri:

1. Grafın tüm düğümlerini içerir.
2. Tam olarak $|V| - 1$ kenar içerir.
3. Dongu içermez (acyclic).

MST Ornegi – Ornek Graf:



Pratik – MST

(a) $V = \{1, 2, 3, 4\}$, $E = \{(1, 2, 5), (1, 3, 3), (2, 3, 1), (2, 4, 4), (3, 4, 2)\}$ için MST'yi bulun. Hangi kenarlar secilir? _____

(b) $|V|$ düğümlü bir spanning tree kaç kenar içerir? Neden? _____

2. Kruskal Algoritması

Kruskal acgozlu (greedy) bir algoritmadır. Kenarlari maliyete gore sıralar, en küçük maliyetli kenardan baslayarak dongu olusturmayanlari MST'ye ekler.

Kruskal Algoritmasi

Calisma Adimlari:

1. Grafın tüm kenarlarını maliyete göre küçükten büyüye sıralanmış.
2. Grafta kullanılmayan en düşük maliyetli kenarı bul.
3. Seçilen kenar bir döngü oluşturmuyorsa ekle.
4. Yayılan ağac $|V| - 1$ kenara ulaşana kadar 2. ve 3. adımları tekrarla.

Union-Find Veri Yapisi:

Kruskal'da iki düğümün aynı bileşende olup olmadığını hızlıca kontrol etmek için Union-Find (Disjoint Set) kullanılır. `find(x)`: x 'in kök düğümünü bulur. `unionSet(x,y)`: iki kümenin birleştirir.

```
static int[] parent;

static int find(int i) {
    if (parent[i] == i)
        return i;
    return parent[i] = find(parent[i]); // path compression
}

static void unionSet(int x, int y) {
    int xroot = find(x);
    int yroot = find(y);
    parent[xroot] = yroot;
}
```

Kruskal MST Ana Kodu:

```
static void KruskalMST(int V, Edge[] edges) {
    Edge[] result = new Edge[V];
    int e = 0;
    parent = new int[V];
    for (int v = 0; v < V; v++)
        parent[v] = v; // her düğüm kendi ebeveyni

    // Kenarları ağırlığa göre sırala
    Arrays.sort(edges, Comparator.comparingInt(edge -> edge.weight));

    int i = 0;
    while (e < V - 1) { // |V|-1 kenar bulana kadar
        Edge next = edges[i++];
        int x = find(next.src);
        int y = find(next.dest);

        if (x != y) { // Döngü oluşturmuyor mu?
            result[e++] = next;
            unionSet(x, y); // Birleştir
        }
        // x == y ise döngü oluşturur, atla
    }
}
```

Ornek – Adım Adım:

Adim	Kenar	Agirlik	Ekle?	Neden?
1	$B - C$	1	✓	Dongu yok
2	$C - E$	1	✓	Dongu yok
3	$A - B$	2	✓	Dongu yok
4	$A - C$	3	×	A, B, C, E zaten bagli – dongu olusur
5	$B - D$	3	✓	Dongu yok, $ V - 1 = 4$ tamamlandi

MST kenarlari: $B-C, C-E, A-B, B-D \Rightarrow$ Toplam maliyet: $1 + 1 + 2 + 3 = 7$

Karmasiklik Analizi:

Kruskal iki ana maliyet icerir:

1. Siralama maliyeti: $|E|$ kenari siralamak $\rightarrow O(|E| \log |E|)$

2. Union-Find maliyeti: Her kenar icin **find** ve **union**: $O(\alpha)$ amortize (α ters Ackermann fonksiyonu, pratikte sabit). $|E|$ kenar icin: $O(|E| \cdot \alpha) \approx O(|E|)$

Siralama	$O(E \log E)$
Union-Find	$O(E \cdot \alpha(V)) \approx O(E)$
Toplam	$O(E \log E)$

Basitlestirme: $|E| \leq |V|^2$ oldugu icin $\log |E| \leq 2 \log |V|$, yani:

$$O(|E| \log |E|) = O(|E| \log |V|^2) = O(|E| \cdot 2 \log |V|) = O(|E| \log |V|)$$

Dolayisiyla:

Sart maliyeti	$O(E \log E)$ $= O(E \log V ^2) = O(4)$
$T(E , V)$	$= E \log E + E \log V $
Odev: her ikisi de gercekten ayni mi?	$ E \log V ^2 \Rightarrow O(E \cdot 2 \log V)$ $= O(E \log V ^2) \Rightarrow O(E \cdot 2 \log V)$
$O(E \log V ^2)$	$\Rightarrow O(E \cdot 2 \log V)$
Sonuc:	$\Theta(E \log V)$

Pratik – Kruskal

(a) $V = \{A, B, C, D, E\}$, agirlikli kenarlar: $(A, B, 4), (A, C, 2), (B, C, 1), (B, D, 5), (C, D, 3), (D, E, 6)$. Kruskal ile MST'yi bulun; her adimda hangi kenari neden sectiginizi yazin. _____

(b) Union-Find olmadan dongu tespiti nasıl yapılırdı? Karmaşıklığı ne olurdu? _____

(c) $|E| = 100$, $|V| = 20$ için $O(|E| \log |V|)$ yaklaşık kaç işlem? _____

3. Kruskal – Avantaj ve Dezavantajlar

Kruskal – Avantaj ve Dezavantajlar

Avantajları:

1. Anlaşılması nispeten kolay.
2. MST bulunmasını garanti eder.
3. Bazı varyantları paralelleştirilebilir.
4. Bağlantısız bileşenlere sahip graflar işleyebilir; bu tür graflar için ayrı ayrı MST bulabilir (her bileşen için ayrı spanning tree).
5. Seyrek graflar için daha hızlı (Prim'e göre).

Dezavantajları:

1. Yoğun graflar için yavaş çalışır ($|E| \approx |V|^2$ olduğunda sıralama pahalı).
2. Dinamik graf güncellemelerini desteklemez (kenar eklenip çıkarıldığında MST'yi yeniden hesaplamak gerekir).

	Kruskal	Prim
Yaklaşım	Acıgözlü (kenar bazlı)	Acıgözlü (dugum bazlı)
Zaman	$O(E \log V)$	$O(E \log V)$ binary heap
Seyrek graflar	Daha hızlı	Biraz yavaş
Yoğun graflar	Yavaş	Daha hızlı (Fib. heap ile)
Uygulama alanı	Genel, dağıtık	Tek kaynak, yoğun

Notlar – Ödev konu başlıkları:

- Prim Algoritması → Backtracking Algoritmaları
- Prim Algoritması → Dinamik Programlama
- Prim Algoritması → Grid Yazılımı

Pratik – Karsilastirma

(a) Seyrek bir grafta ($|E| \approx |V|$) Kruskal toplam karmasikligi nedir? Yogun grafta ($|E| \approx |V|^2$)? _____

(b) Prim ile Kruskal'in temel yaklasim farki nedir? _____

Ozet:

- MST: $|V|$ dugum, $|V| - 1$ kenar, dongu yok, toplam agirlik minimum.
- Kruskal: kenarlari sirala, dongu olusturmayan en ucuz kenari ekle.
- Union-Find ile dongu tespiti: $O(\alpha) \approx O(1)$ amortize.
- Toplam karmasiklik: $\Theta(|E| \log |V|)$.
- Seyrek graflar icin Kruskal, yogun graflar icin Prim tercih edilir.